Roof

Warranty & Receipt Vault | Project definition and architecture proposal | React + Node.js/Express + MongoDB

The product: a responsive personal web app that keeps purchase records and receipt files in one private account, calculates warranty dates, shows what is expiring, emails reminders, and stores service-center contacts.

01 Refined idea and scope

A user registers, adds a purchased product, enters the warranty terms, and optionally attaches a receipt. The app shows the coverage status and provides the receipt and support contact when the user needs them. This directly addresses the SRS problem: lost proof of purchase and missed warranty deadlines.

MVP capability	What the finished app must do
1. Accounts and isolation	Register, log in/out; every product, receipt, contact and reminder belongs to its authenticated owner.
2. Product and warranty	Create, read, edit and delete products; calculate an end date from start date and duration; allow correction for manufacturer terms.
3. Receipt vault	Attach optional PDF/image receipts; validate, privately store, preview and download them.
4. Dashboard and search	Show totals for active, expiring soon, expired and incomplete warranties; search name/brand and filter status.
5. Email reminders	Choose reminder offsets; schedule, send, retry and record outcomes without duplicate sends.
6. Service contacts	Save and view product-linked service-center name, phone, address and website.

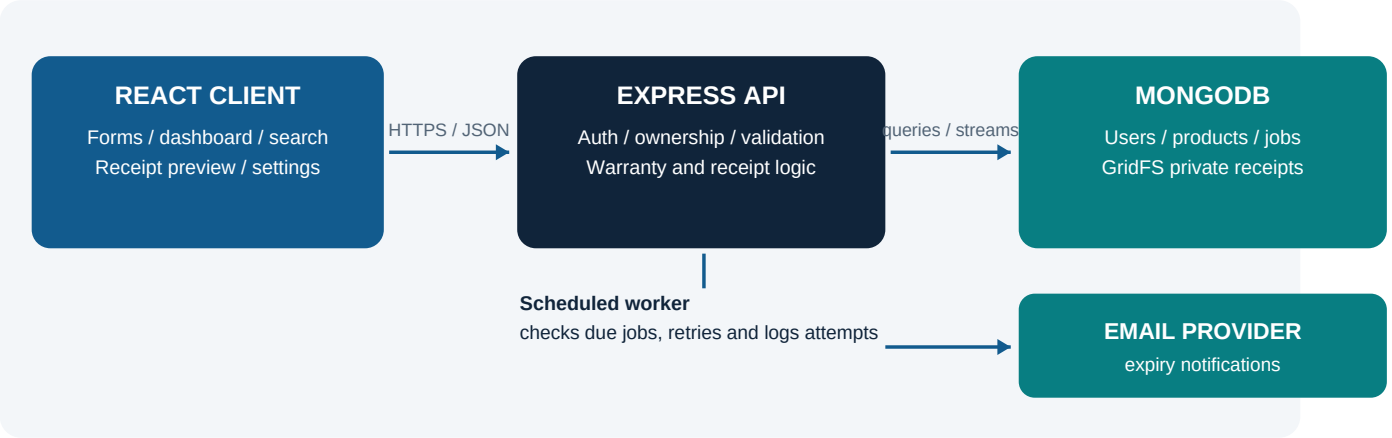
Validated design decisions

- **MongoDB is fixed** as the database, replacing the SRS's undecided database; MongoDB GridFS stores private receipt bytes.
- **Email is the MVP notification channel.** The SRS mentions channels generally, while the requested build specifies email.
- **Receipt upload is optional.** A product can be saved without a receipt. A failed upload is reported and can be retried.
- **No admin interface in the MVP.** The SRS describes an administrator but gives no required admin workflows. Operational monitoring remains server-side.

Outside this release: OCR, automatic claim filing, family sharing, retailer integrations, and push notifications. Warranty status is an aid; manufacturer terms remain authoritative.

02 System architecture

One React browser client talks to an Express REST API. The API owns business rules and authorization. MongoDB stores structured records and GridFS receipt files. A Node.js worker handles scheduled email jobs; an external email provider delivers them.



Layer	Responsibility and implementation
React + JavaScript	Responsive routes for auth, dashboard, product form/detail, receipt preview, search, reminder settings and contacts. Show loading, validation and failed-operation states.
Node.js + Express	REST routes, schema validation, password hashing, session handling, user-scoped queries, warranty calculation, upload/download streaming and error responses.
MongoDB	Collections and indexes for users, products, receipts, contacts, preferences and jobs; GridFS for private receipt content.
Background worker	Find due jobs, claim them atomically, send through the configured email provider, log attempts, and retry failures with a limit.

Privacy and reliability rules

• Hash passwords (Argon2id or bcrypt); use server-managed sessions in HttpOnly, Secure cookies over HTTPS. Protect write requests against CSRF.

• Never trust a client-supplied user ID. Scope every database query to the session's user ID and check ownership before streaming a receipt.

• Allow only approved PDF/JPEG/PNG types and a documented size limit; inspect file content, generate internal storage names, and never expose GridFS directly.

• Keep secrets in environment variables. Back up MongoDB, including GridFS; record failed uploads and email attempts. Use indexes for user-scoped lists and due jobs.

Deployment shape

Host the React build as static assets, run the Express API and reminder worker as separate Node processes, and connect them to a persistent MongoDB instance. Configure an email provider, HTTPS, environment secrets, backups and a recovery check. Local development can run the same three services with a test mailbox.

03 Data model and warranty rules

The supplied ER diagram is a useful starting point. These additions make the six requested capabilities implementable and keep ownership explicit.

MongoDB collection	Essential fields / relationship
users	_id, name, unique email, passwordHash, timeZone, createdAt. One user owns many products.
products	_id, userId, name, brand, model, purchaseDate, warrantyStartDate, warrantyMonths, warrantyEndDate, endDateSource, terms, createdAt/updatedAt.
receipts	_id, userId, productId, gridFsFileId, originalName, mimeType, size, uploadedAt. A product may have zero or more receipts.
serviceCenters	_id, userId, productId, name, phone, address, website. A product may have zero or more contacts.
reminderPreferences	userId, enabled, offsetsDays (for example 30, 7, 1), updatedAt. One setting per user.
reminderJobs	userId, productId, warrantyEndDate, offsetDays, scheduledFor, status, attempts, lastError, sentAt; unique key prevents duplicate jobs.

Warranty calculation contract

- The user enters a start date and a duration in whole months; start date defaults to purchase date but can be changed.
- The calculated **last covered day** is the calendar-month anniversary minus one day. For short target months, clamp the anniversary to the month's last day. Show the computed result before save.
- Users can enter the manufacturer-stated end date instead. Store whether the date was calculated or manually entered, plus any terms/notes. Do not invent an end date when data is missing.
- Treat dates as date-only values. In the user's configured time zone, status is **active** through the end date, **expiring soon** in the final 30 days, **expired** after it, or **incomplete** without an end date.

Data integrity and lifecycle

Validate dates, duration, file type and size at the API. On warranty edits, cancel obsolete pending jobs and regenerate future jobs. On product deletion, remove linked receipts/GridFS files, contacts and pending jobs; keep only minimal delivery audit entries if a retention policy requires them. A cleanup task reconciles failed partial deletions.

Indexes: unique users.email; products(userId, name/brand), products(userId, warrantyEndDate); receipts(userId, productId); contacts(userId, productId); jobs(status, scheduledFor) plus a unique job identity on productId, warrantyEndDate and offsetDays.

04 User flows, API and completion criteria

Core flows

• **Add product:** React form → API validation → warranty calculation → owner-linked product saved. A receipt may be uploaded now or later. This refines the supplied activity diagram.

• **Retrieve receipt:** user opens a product → API checks product and receipt ownership → GridFS stream returns a preview or download.

• **Send reminder:** worker claims a due job → verifies current product expiry and preference → emails the user → records success/failure and bounded retry.

Proposed REST surface

Area	Endpoints (all private routes require authentication)
Auth	POST /api/auth/register, /login, /logout; GET /api/auth/me
Products	GET /api/products?search=&status=; POST /api/products; GET/PATCH/DELETE /api/products/:id
Receipts	POST /api/products/:id/receipts; GET /api/receipts/:id/file; DELETE /api/receipts/:id
Dashboard	GET /api/dashboard/summary (counts and upcoming expiries)
Reminders	GET/PUT /api/reminder-preferences; GET /api/reminders/history
Contacts	GET/POST /api/products/:id/service-centers; PATCH/DELETE /api/service-centers/:id

Definition of a working college-project release

• Two test users can sign in; neither can read or change the other's product, receipt, contact or reminder by altering an ID.

• Product CRUD, valid/invalid receipt upload, preview/download, date calculation, manual correction, dashboard counts, search and filters work end to end.

• A test email is sent for a due warranty; a failed send is recorded and retried; duplicate sends are prevented after a worker restart.

• Deleting a product removes linked private data. Missing warranty details remain incomplete, and invalid inputs produce clear errors.

Build sequence

1) MongoDB schemas, auth and ownership → 2) product/warranty API and React screens → 3) GridFS receipts → 4) dashboard/search and contacts → 5) worker/email and integration tests → 6) deployment, backups and demo data.

Source alignment: Based on the supplied SRS (draft v1.0), UC-01 to UC-03, activity diagram and ER diagram. This document is the refined build specification; it does not claim the software is already implemented.